

UNIDADE 6 · REDES NEURAIS ARTIFICIAIS

O Neurônio Artificial e a Arquitetura de Rede Densa

Encontro 1, Bloco 1 · A ponte com tudo que vocês já sabem

Um problema que vocês ainda não sabem resolver

Imaginem 600 alunos fictícios: cada um com um número de horas de estudo e um nível de 'facilidade de aprendizado' (0 a 1). Queremos prever se cada um foi aprovado ou não.

A regra por trás dos dados (que os dados escondem, mas existe) é: quem estuda pouco tem baixa chance; quem estuda uma quantidade certa tem a MAIOR chance; quem estuda demais volta a ter baixa chance.

Nenhum algoritmo que vocês já aprenderam (Regressão Logística, KNN, Árvore, Random Forest) foi desenhado pensando nesse tipo de padrão especificamente.



Por que não usar Regressão Polinomial?

Já sabemos que Regressão Polinomial resolve curvas. Testamos, de verdade, em vários graus:

Grau	Acurácia	Features geradas
1 (reta comum)	77,5%	2
2	79,2%	5
5	83,3%	20
10	84,2%	65
Rede Neural (4 neurônios)	88,3%	—

Com só 10 features originais, grau 3 já gera 285 features. Com 20, gera 1.770 -- não escala.

Scikit-Learn ou Keras? As duas fazem a mesma coisa

Existem várias bibliotecas pra construir redes neurais -- as mais usadas no mercado são Scikit-Learn (MLPClassifier/MLPRegressor) e Keras/TensorFlow.

Scikit-Learn: sintaxe igual à que vocês já usam (.fit(), .predict(), .score()). Keras: sintaxe própria, construída 'camada por camada' -- padrão da indústria pra redes maiores (imagem, texto).

Hoje usamos Scikit-Learn -- a mesma família de ferramentas que já dominam. Semana que vem, a mesma rede, escrita em Keras.



A base de dados que vamos usar hoje

600 alunos fictícios, 1 arquivo só (dataset_alunos.csv), com 2 entradas (horas_estudo, facilidade_aprendizado) e 2 colunas-alvo: nota (0 a 10) e aprovado (0 ou 1).

aprovado NÃO é sorteado à parte -- é só nota ≥ 6 , direto. As duas colunas contam a mesma história, uma como número, outra como categoria.

Por isso dá pra usar o MESMO arquivo tanto pra classificação (aprovado) quanto pra regressão (nota) -- só troca a coluna-alvo.



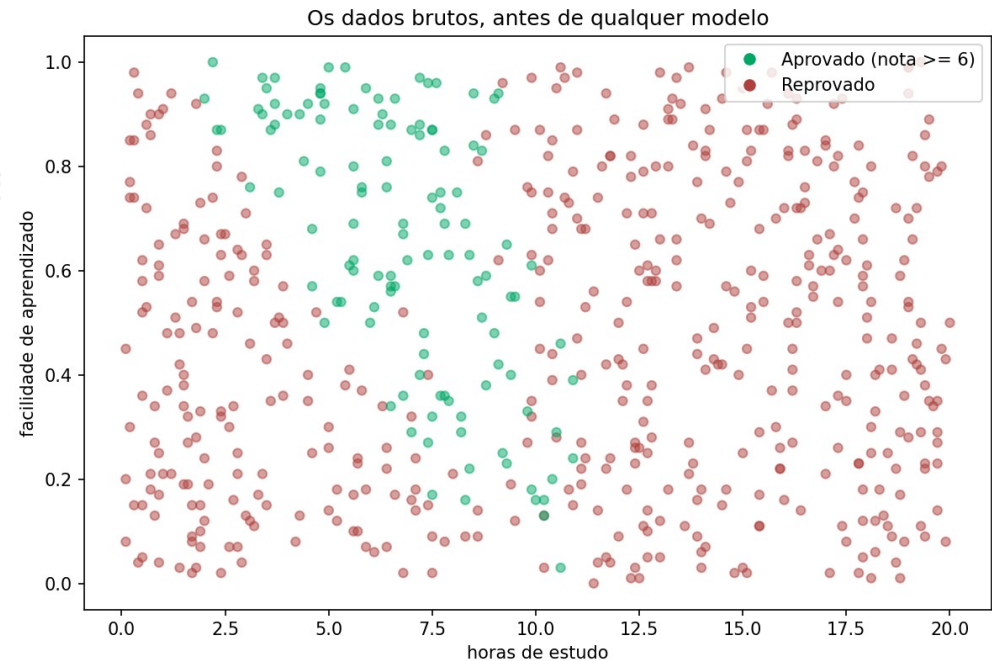
Gerando e olhando os dados (1 arquivo, 2 alvos)

```
import pandas as pd

df = pd.read_csv("dataset_alunos.csv")
print(df.head())

#  horas  facilidade  nota  aprovado
#    7.5      0.17    6.3      1
#   19.0      0.28    0.0      0
#   14.6      0.18    0.0      0

# aprovado = (nota >= 6).astype(int)
# -- ja vem pronta no arquivo
```



Relembrando: a fórmula que vocês já sabem

Na Regressão Logística, com 2 features, vocês já calculavam:

$$z = w1 \times \text{feature1} + w2 \times \text{feature2} + w0$$

$$\text{probabilidade} = \text{sigmoide}(z)$$

Peso (w) e intercepto (w0) -- isso é EXATAMENTE o que um neurônio artificial calcula, só com nomes diferentes.



O neurônio artificial: mesma conta, nomes novos

peso das horas = 2, peso das faltas = -3, viés (mesmo que w_0) = 1. Aluno com 4h de estudo e 1 falta:

$$\begin{aligned}z &= (2 \times 4) + (-3 \times 1) + 1 \\z &= 8 + (-3) + 1 \\z &= 6\end{aligned}$$

Só isso: multiplicar cada entrada pelo peso, somar tudo, somar o viés uma vez só. Nada de novo.



⚠ Cuidado: só existe 1 z, nunca um por entrada

Um erro comum: pensar que cada entrada (horas, faltas) teria seu próprio 'z' e seu próprio viés.

Não é assim -- o neurônio pega TODAS as entradas de uma vez, e produz UM ÚNICO z. Só existe 1 viés, somado 1 vez, no final.

z1 pra notas e z2 pra faltas, cada um com viés próprio, ESTÁ ERRADO. É sempre 1 z só, somando tudo.



Um segundo neurônio: mesmos dados, pesos diferentes

Neurônio 2, com pesos diferentes do Neurônio 1, olhando pras MESMAS entradas (4h, 1 falta):

$$z2 = (0.5 \times 4) + (-1 \times 1) + 0$$

$$z2 = 2 + (-1) + 0$$

$$z2 = 1$$

2 cálculos de regressão, em paralelo, cada um com seu próprio peso -- nada de 'engajamento' ou rótulo próprio.



Por que eu ia querer 2 retas calculando a mesma coisa?

Sozinho, cada neurônio ainda é só uma reta simples -- exatamente o tipo de coisa que uma Regressão Logística já faz.

Mas uma reta só sabe fazer UM corte: 'acima de tal valor = uma coisa, abaixo = outra'. Ela nunca consegue representar um 'monte' (sobe e depois desce) -- isso exige pelo menos 2 cortes diferentes, combinados.

A vantagem só aparece quando COMBINAMOS os resultados de vários neurônios -- isso ainda não fizemos.



De onde vêm os pesos? Você não escolhe -- o treino decide

Nenhum peso é escrito por uma pessoa. Todos começam ALEATÓRIOS, sem sentido -- e são ajustados sozinhos, durante o treino, a cada rodada.

Se, na prática, um neurônio ajuda a reduzir o erro final, seu peso cresce. Se não ajuda, o peso é empurrado pra perto de zero -- exatamente a mesma busca iterativa (Gradiente Descendente) que já vimos na Regressão Logística.

É a mesma lógica de 'a Árvore testa várias perguntas e escolhe a melhor' -- só que aqui é ajuste de números, não escolha de pergunta.

O que é uma 'função de ativação'

É só isso: depois de calcular o z , você faz MAIS UMA COISA com esse número, antes de usar ele. Essa 'coisa a mais' é a função de ativação.

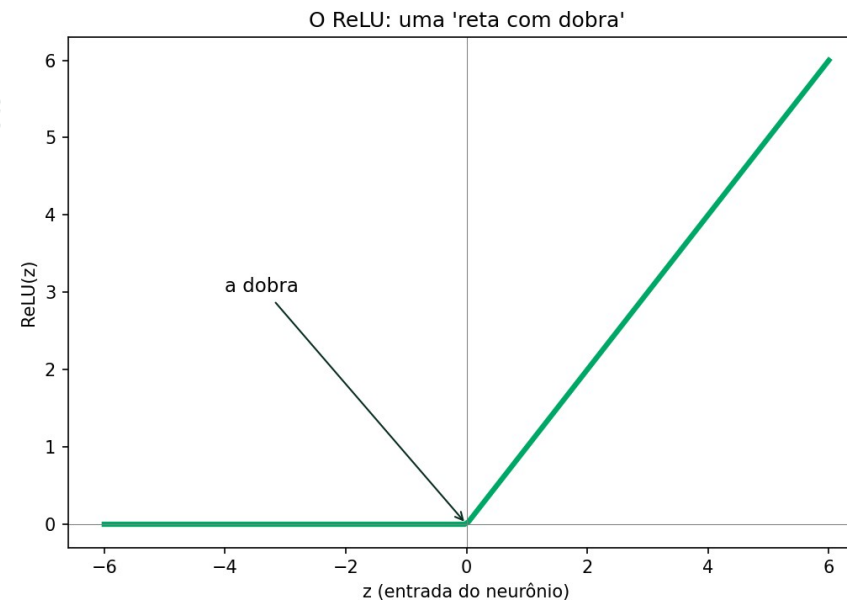
Vocês já usam uma: a sigmoide! Pega o z e devolve algo entre 0 e 1. 'Função de ativação' é só o nome genérico da categoria -- sigmoide é UM exemplo dela.

Sigmoide não é a única opção. A mais comum dentro das camadas ocultas tem outro nome: ReLU.

ReLU: a função de ativação mais usada nas camadas ocultas

Regra simples: negativo vira zero, positivo passa direto. Isso cria uma 'reta com dobra' (uma quina no gráfico).

```
def relu(z):  
    return max(0, z)  
  
print(relu(6))      # 6  (positivo, passa direto)  
print(relu(-5.6))   # 0  (negativo, vira zero)
```



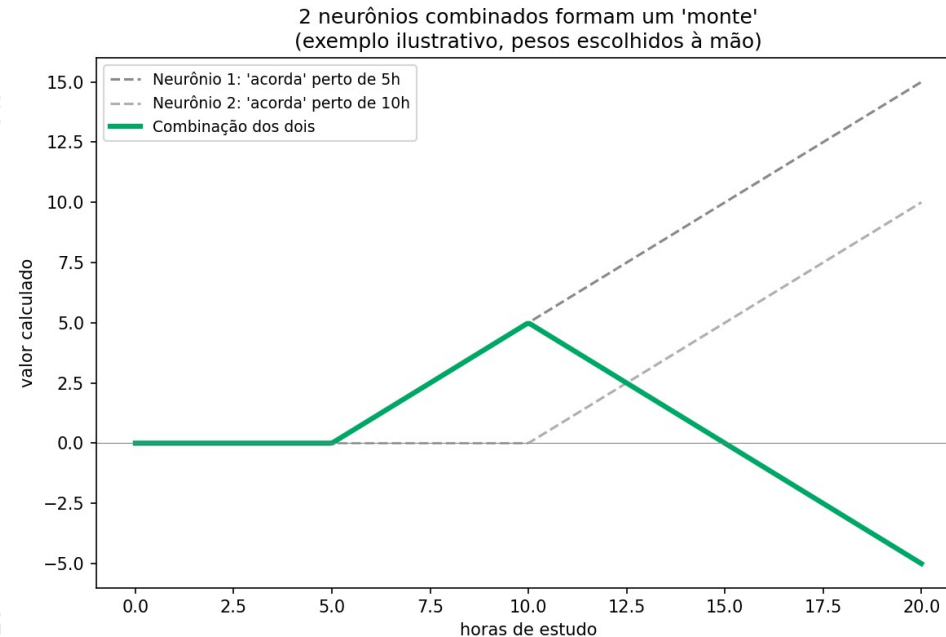
2 neurônios combinados formam um 'monte'

Pesos escolhidos à mão, só pra ilustrar o mecanismo. Repare: nenhuma reta sozinha faz esse sobe-e-desce.

```
# Neurônio 1: 'acorda' perto de 5h
a1 = relu(1.0 * (horas - 5))

# Neurônio 2: 'acorda' perto de 10h
a2 = relu(1.0 * (horas - 10))

# Neurônio de saída: combina os dois
z_final = 1.0*a1 - 2.0*a2
```



Um 3º neurônio: combinando a1 e a2

O terceiro neurônio recebe a1 e a2 como ENTRADAS -- com pesos totalmente novos, próprios dele:

$$\begin{aligned}z_{\text{final}} &= (0.5 \times a1) + (0.3 \times a2) + (-2) \\z_{\text{final}} &= (0.5 \times 6) + (0.3 \times 0) + (-2) \\z_{\text{final}} &= 1\end{aligned}$$

peso_a e peso_b não têm nada a ver com os pesos dos neurônios 1 e 2 -- são pesos NOVOS, de um neurônio novo.

O que é uma camada

Lembra dos neurônios 1 e 2? Os dois recebiam as MESMAS entradas (horas, faltas), calculando cada um o seu z , em paralelo, sem um depender do outro.

Isso é uma camada: um grupo de neurônios que recebem as mesmas entradas, e calculam tudo ao mesmo tempo, cada um com seus próprios pesos.

No nosso exemplo: Camada 1 = neurônios 1 e 2 (produzem a_1 , a_2). Camada 2 = o neurônio que combina a_1 e a_2 .



Por que ter mais de uma camada

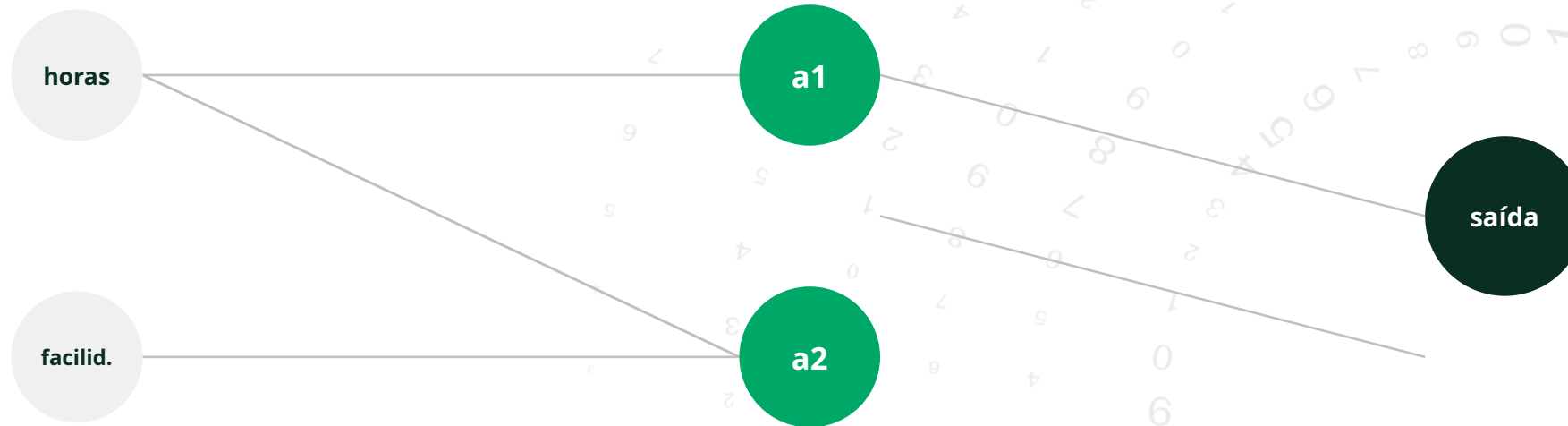
Cada neurônio da Camada 1, sozinho, só cria UMA dobra em relação às entradas originais. Uma dobra só não representa um 'monte' -- isso precisa de 2 dobras trabalhando juntas.

A Camada 2 pega as dobras já calculadas (a_1 , a_2) e as combina numa forma nova, mais complexa do que qualquer dobra sozinha conseguiria.

Mais camadas = combinar combinações anteriores, em formas cada vez mais elaboradas. É por isso que 'rede profunda' significa 'muitas camadas empilhadas'.



Juntando tudo: rede densa (Multilayer Perceptron)



Camada de Entrada (features) -> Camada Oculta (aprende padrões, com ReLU) -> Camada de Saída (decisão, com sigmoide)

'Densa' = todo neurônio de uma camada se conecta com TODOS da próxima.

Quantos neurônios? Quantas camadas? Você decide

Assim como K (KNN), max_depth (Árvore) e n_estimators (Random Forest), a quantidade de neurônios por camada e de camadas é uma escolha SUA, antes de treinar -- não algo que o algoritmo decide sozinho.

Poucos neurônios: rede simples demais (underfitting). Neurônios demais: risco de decorar o treino (overfitting) -- o mesmo dilema de sempre, com outro nome.

No Scikit-Learn, o número de repetições do treino se chama max_iter. No Keras (semana que vem), o mesmo conceito se chama epochs.

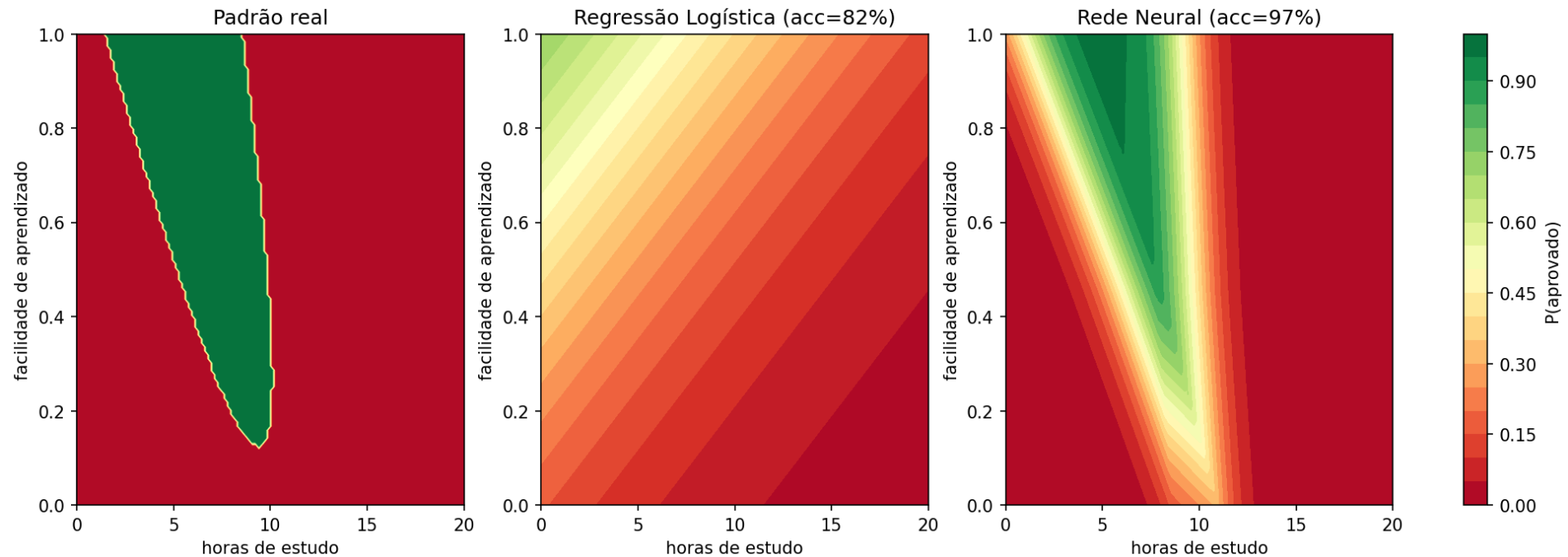
Classificação ou regressão? Só a última camada muda

Tudo que vimos até aqui (neurônios, ReLU, camadas ocultas) é IDÊNTICO nos dois casos -- classificação e regressão.

Classificação: última camada usa sigmoide, virando probabilidade (0 a 1). Regressão: última camada NÃO usa nenhuma ativação -- o z final É a própria previsão, sem espremer em nada.

O 'corpo' da rede é sempre o mesmo. Só a última peça do quebra-cabeça muda, dependendo do que você quer prever.

Resultado: classificação (aprovado ou não)

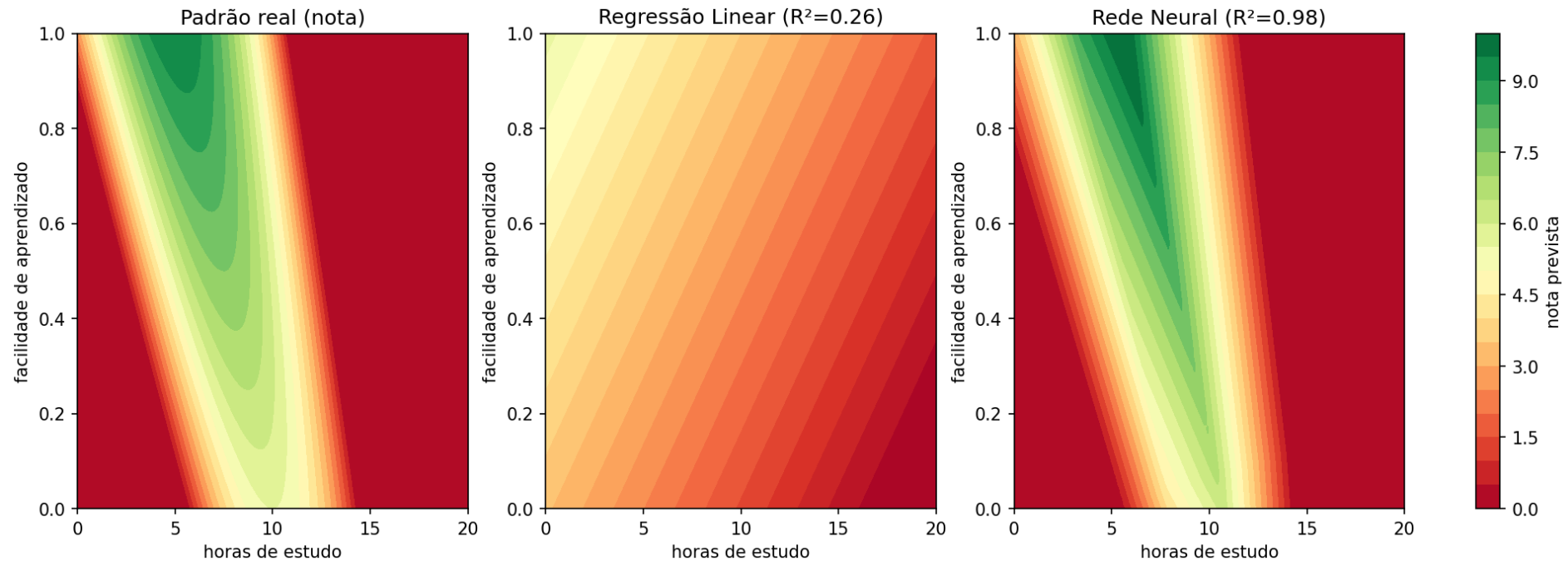


```
df = pd.read_csv('dataset_alunos.csv') # 1 arquivo so
X = df[['horas_estudo', 'facilidade_aprendizado']]
y = df['aprovado'] # coluna derivada de: nota >= 6

log_reg = LogisticRegression().fit(X_train, y_train)
mlp = MLPClassifier(hidden_layer_sizes=(4,), activation='relu').fit(X_train, y_train)

# Regressao Logistica: 82% | Rede Neural: 97%
```

Resultado: regressão (nota exata) -- mesmo arquivo



```
df = pd.read_csv('dataset_alunos.csv') # MESMO arquivo
X = df[['horas_estudo', 'facilidade_aprendizado']]
y = df['nota'] # a coluna continua, sem limiar

lin_reg = LinearRegression().fit(X_train, y_train)
mlp_reg = MLPRegressor(hidden_layer_sizes=(6,), activation='relu').fit(X_train, y_train)

# Regressao Linear: R2=0.26 | Rede Neural: R2=0.98
```

Voltando à pergunta: por que não Regressão Polinomial?

Vimos no início: Polinomial melhora (77,5% -> 84,2%), mas nunca alcança a rede (88,3%) -- e paga um preço crescente de features (65 features só pra 2 entradas, no grau 10).

A rede aprendeu uma fronteira tão boa quanto (ou melhor) SEM precisar que a gente escolhesse grau nenhum -- ela descobre a complexidade certa sozinha, ajustando pesos.

Esse é o argumento central da aula de hoje: redes neurais automatizam a busca por não-linearidade que, com Polinomial, teríamos que adivinhar manualmente.



FIM DO BLOCO 1 -- TEORIA

De um neurônio a uma rede -- a ponte está feita

Neurônio = Regressão Logística. Pesos vêm do treino, não de escolha manual. ReLU cria as 'dobras'. Camadas combinam essas dobras em formas complexas. Só a última camada decide classificação ou regressão.

A seguir, depois do intervalo: mão na massa, com Scikit-Learn, no dataset de vocês.

